

ZaraiLink: Demo

Brief

A short, accurate account of what this repository is, what it proves, and where it is weak.

The two things that must be said first

The data is fabricated. Every company, contact, phone number, price and shipment figure in this repository is invented. No real trade data appears anywhere, and no code here makes a network request. This is an imitation of another application's screens, not that application.

ZaraiLink is a team project. Three people built it: Salman Adnan led it, and Umar Kashif and Fahad Nadeem wrote real, substantial parts. The real system is private and none of its source is in this repository. This demo is Salman's own work; the product it depicts is not his alone.

1 The problem this repository solves

ZaraiLink is an agricultural trade-intelligence platform: a Django and DRF backend, a React frontend, a natural-language search pipeline, hybrid retrieval, and a learning-to-rank model over Pakistani customs data. It is private, and it cannot be made public by one author acting alone.

That creates a real bind. A hiring panel cannot be handed the source, and a screenshot proves nothing. The answer taken here is to rebuild the *screens* against *invented* data. The visual design language and the interaction patterns are things any browser loading the real app already downloads and renders. The private parts stay out.

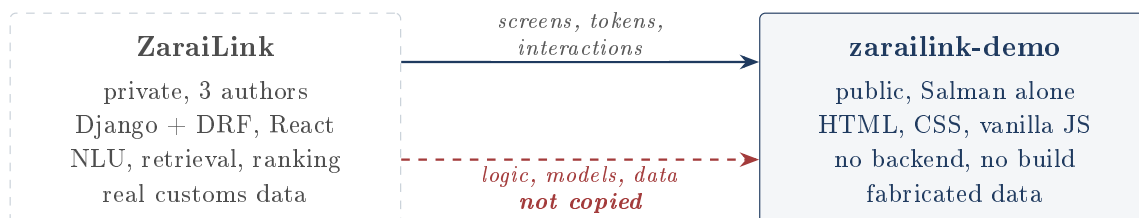


Figure 1: The demo copies the surface and deliberately copies none of the private logic.

2 What it is, technically

Five files, about 1,100 lines, no framework and no build step. What sits in the repository is exactly what the browser runs; you can open `index.html` from a file path, offline, and it works.

File	Role	Lines
<code>app.js</code>	Routing, filtering, unlocking, all rendering	399
<code>styles.css</code>	Colour tokens and component layout	369
<code>index.html</code>	Six screens as static markup	271
<code>demo-data.js</code>	The fabricated dataset	70
<code>icons.js</code>	A 20-entry SVG icon lookup	34

Six screens live in `index.html` at once. CSS hides all but the one carrying an `is-active` class, and navigation is just moving that class. All behaviour hangs off a **single** click listener on `<body>` that inspects what was clicked.

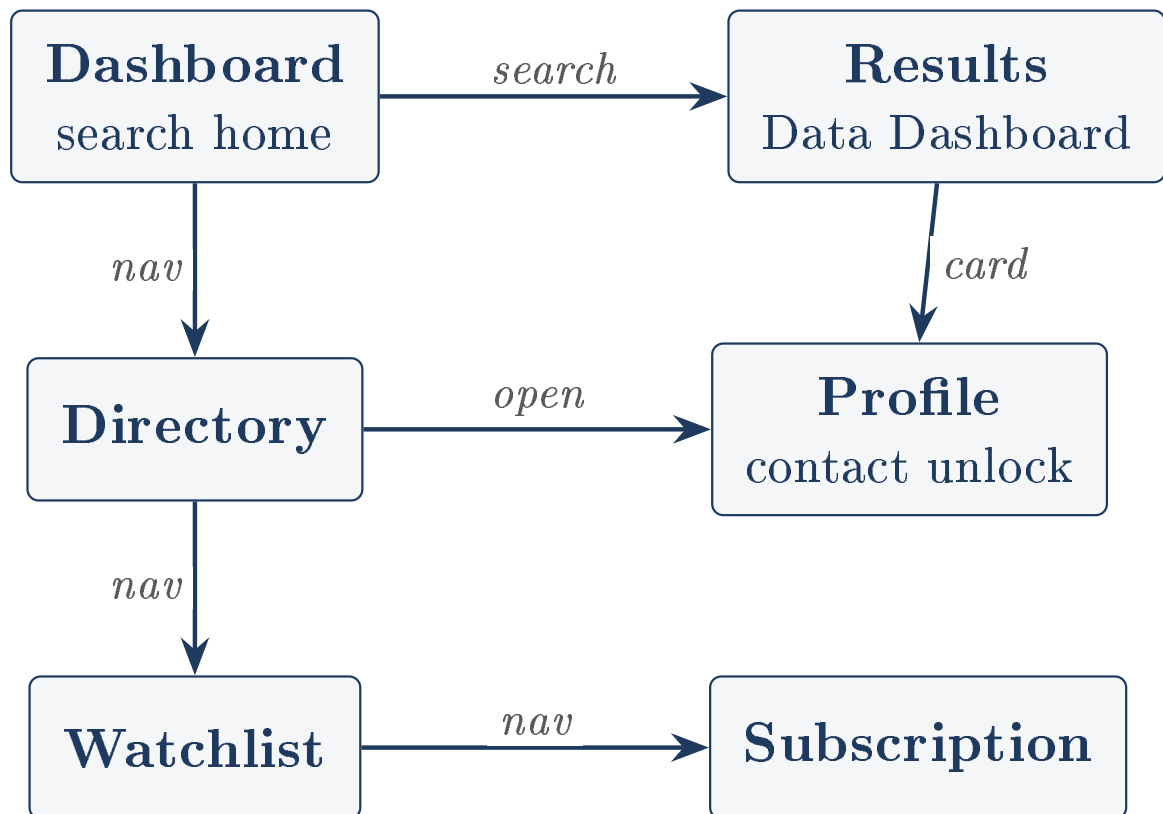


Figure 2: Six screens, one page. No URL ever changes.

Why event delegation is the right call here

Every grid on these screens is drawn by writing an HTML string into `innerHTML`, which destroys and recreates the elements. Any listener attached directly to a button would be thrown away on every render. Listening once on `<body>` and using `e.target.closest(...)` to find what was hit means re-rendering is free and breaks nothing. It is the best structural decision in the codebase.

3 What actually works

- **Per-contact unlocking.** The real application unlocks contacts one at a time, not per company, and the demo reproduces that faithfully. Each contact carries its own `unlocked` flag; a confirm modal stashes the pending target, re-checks the token balance at confirm time rather than trusting the earlier check, spends one token, flips exactly one contact, and re-renders from the model. This is the demo's centrepiece and it is correct.
- **Masking.** Locked phone and email values are rendered mangled (first character, then bullets, domain preserved) and blurred with CSS, then flip to their real fabricated values only for the specific contact that was paid for.
- **Seniority badges.** A short ordered chain of regular expressions turns a job title into one of five badges. It is genuinely reading the data, not hard-coded.
- **The directory search box.** Filters company names case-insensitively on every keystroke, with an automatic empty state.
- **The honesty scaffolding.** A yellow banner on every screen, a footer disclaimer, `noindex` in the head, all domains under the reserved `.example.com`, and the word "fabricated" printed into the results screen itself.

- **No build step, no network, no secrets.** A scan for keys, tokens, passwords and connection strings returned nothing, as expected for a codebase that talks to nobody.

4 Where it is weaker than it looks

This section is the point of the document. Everything below was verified by reading the source and, where it involved logic, by executing it.

4.1 Two real bugs

The category payroll advertises 500 tokens and charges 5

The button reads “Unlock for 500 Tokens”. The code guards on `tokenBalance < 5` and then deducts 5. The user starts with 7, so the button *works*, which is precisely why the bug survives: nothing looks broken. Against the plan grants (Starter gives 20 tokens per month) a 500-token unlock would be absurd, so the label is almost certainly the typo. The cost 5 is also a magic number written twice, with the advertised price a third, unrelated literal.

Un-starring on the Watchlist screen does nothing visible

The star handler unconditionally calls `renderDirectory()` and never `renderWatchlist()`. Both grids share the same card template, so the Watchlist has star buttons. Click one there and the model updates correctly, but the re-render lands on a hidden screen; the grid in front of you never changes. Navigate away and back and the change appears, which is the signature of this class of bug.

This is the most likely defect for an interviewer to hit by accident, because starring things and then going to the watchlist to unstar one is the obvious way to explore the feature.

4.2 The demo is thinner than its screens imply

- **Search does not filter.** The query is used only as a heading. Search “PVC Resin” and you get the same four dextrose and urea companies as searching “dextrose”. Any numeric query hard-codes the heading “HS 1702.3000: Dextrose” regardless of what was typed.
- **Four trade-direction pills, two behaviours.** “Foreign Suppliers” and “Foreign Buyers” have *identical* predicates (`country !== "Pakistan"`), as do the two Pakistani pills. Only the country axis does anything; the buyer/supplier axis is decoration. Verified by running them: 3, 3, 1, 1 results, the same 3 and the same 1.
- **The Trade Directory has one mode wearing two hats.** Switching to “Find Buyers” rewrites the heading and the lede and shows the same four companies. There is no buyer/supplier distinction in the data. The empty state still says “No suppliers found.”
- **The shipment chip contradicts the data.** It shows `COMPANIES.length * 14`, which is 56. The dataset’s own `shipments` fields sum to 125.
- **Buying a plan is an unconditional faucet.** No payment, no current-plan state, no limit. Click Enterprise ten times for 3,000 tokens.
- **The world is small.** Four companies, five contacts, one of them already free. With a starting balance of 7 a user can unlock everything and still have 3 left, so the scarcity the screens dramatise never bites.

4.3 Dead controls, the highest-cost defect here

Six visible things respond to the mouse and do nothing:

Control	Reality
Import/Export toggle	<code>currentScope</code> is written on every click and read nowhere
Theme toggle (moon)	Styled, <code>aria-labelled</code> , no listener, and no dark theme exists
“I want to buy” / “I want to sell”	No <code>data-view</code> , no handler. The two pills beside them <i>do</i> navigate, which makes the dead pair harder to spot
User menu	<code>cursor: pointer</code> and a hover state advertise a drop-down that does not exist
“Refine by Product” checkboxes	Real, correctly computed counts; no listener. Convincing precisely <i>because</i> the counts are right

For a demo whose entire purpose is to be clicked by a stranger, each dead control is an invitation to find the seam. A demo with fewer controls that all work is strictly stronger than one with more controls where half are scenery.

4.4 One feature the README claims that the code cannot do

“Product Search” mode is unreachable

`detectMode(query, scopeSelected)` returns `"product"` only when `scopeSelected` is falsy, but **both** call sites pass the literal `true`. So a query starting with a digit gives `"hscode"`, any other non-empty query gives `"ai"`, and the empty case is caught by an early return that clears the pill entirely. The green “Product Search” pill can never render. Confirmed by executing the function.

The README states the pill “flips between HS Code Mode, Product Search, and AI Query Mode”. Two of the three are real. Passing `currentScope` instead of `true` would not fix it either, since it is initialised to `"import"` and is always truthy; the intended design presumably starts the scope unselected.

4.5 Where the README and the code disagree

The README describes a font strategy the code abandoned

The README says the demo “falls back to the closest system font instead of fetching” Inter externally. It does not. It ships `assets/fonts/inter-variable.woff2` and uses real Inter. The git history explains the drift: the commit that self-hosted the font came after the README paragraph was written, and the paragraph was never updated. `styles.css`’s own header comment is correct. **The code wins; the README is stale.**

This matters more than a normal doc bug. A project sold on honesty about what is real cannot afford a README that overstates its own code, however innocently it drifted.

4.6 Dead data

`HS_TREE` (a real fragment of the product’s model: code, name, category) is declared and referenced nowhere; the Dashboard hard-codes the example `1702.3000` in HTML instead of reading it. `company.website` and `company.last_active` are populated on every company and rendered never. Ten of the twenty icons are never called, and several are duplicated as inline markup in `index.html`, so one icon set is maintained in two places.

4.7 Security

Nearly everything renders through `innerHTML`. `escapeHtml` handles `&`, `<` and `>` but **not quotes**, and several values land inside HTML attributes where a quote would break out.

This is **not exploitable**, and the reason is worth stating precisely: every string in the application is a hard-coded literal. There is no user-supplied data, no network input, no URL parameter, no storage, and the two text inputs are read but never rendered back as HTML. The attack surface is empty. It is not a vulnerability; it is a habit that would become one the instant this code met real data, which is exactly what the real ZaraiLink does.

Likewise the paywall blur is pure CSS, so the values sit in the DOM for anyone who opens the developer tools. Harmless here because everything behind it is invented, but a real paywall must withhold data at the server and never send it.

5 What is claimed but cannot be checked here

Three claims rest outside this repository

1. **“64 of 97 commits”** (README). The private repository’s git history is not here. This is the owner’s to stand behind; an interviewer who asks should be pointed at the private repository.
2. **That the screens match the real components.** The README and `styles.css` cite specific real files (`Dashboard.js`, `DataDashboard.js`, `CompanyProfile.js`) and state the colours were copied 1:1 from the real `index.css`. None of those files are in this repository, so the fidelity claim is **inferred, not verified**.
3. **That masking mirrors the real `maskValue()`.** Same reason. Inferred.

Separately, the font file is 48 KB, which is small for a full variable Inter, so it is *probably* a subset. That is **inferred from file size** and was not confirmed by inspecting the font tables.

Note also that this demo is evidence of UI reconstruction skill and product judgment. It is **not** evidence that the real backend works, because none of it is here.

6 Licence and deployment

`netlify.toml` is two lines (`publish = "."`) with no build command, because there is nothing to build. `index.html` carries `noindex`, correctly: a page full of invented companies and invented contact details should not appear in a search for those names.

The licence is **PolyForm Strict 1.0.0**, which is source-available, *not* open source: it permits reading and nothing else. For portfolio work that is coherent, and especially so here, where the underlying product is a private team project. It should not be described as open source. The copyright line reads “Copyright 2026 Salman Adnan” on a repository whose subject matter belongs to three people; the demo’s code is Salman’s alone, so that holds, but the distinction between “this demo’s code” and “ZaraiLink” is doing real work in that line.

7 The highest-value hour of work

Not a new feature.

1. **Fix the 500/5 token label and the watchlist re-render.** Both are one-line fixes; both are things a clicking interviewer will find.
2. **Correct the README’s font paragraph and soften the three-modes claim.**

3. **Wire up or delete the dead controls.** Deleting them costs nothing and removes six chances for the demo to look broken rather than scripted.

The README’s “What I’d do differently” proposes adding the Trade Intelligence cluster (link prediction, node2vec, the GNN work). That is the right *ambition* and the wrong *next step*: adding a sixth screen to a demo with six dead controls widens the seam rather than closing it.

8 Glossary

Backend / frontend

The frontend runs in the browser and draws what you see; the backend runs on a server and holds the data. This demo has only a frontend, and it asks nobody anything.

Build step

A program that translates source the browser cannot run (JSX, TypeScript, Sass) into plain HTML, CSS and JS. This demo has none, which is why it still works when opened from a file path with no toolchain installed.

DOM

The browser’s live, in-memory tree of the page. JavaScript changes what you see by changing this tree.

Event delegation

Listening for clicks once at the top of the page and working out what was hit, instead of attaching a listener to every button. Relies on *bubbling*, the browser’s habit of firing an event on an element and then on each of its ancestors.

HS code

The Harmonized System number for a traded good. Customs records are indexed by it. 1702.3000 is dextrose.

Regular expression

A small pattern language for matching text. `/^\d/` asks “does this start with a digit?”

SVG

Pictures drawn with maths rather than pixels, so they stay sharp at any size. `currentColor` means “whatever the CSS text colour is here”, which lets one icon definition work on a dark navbar and a white card.

Variable font

One font file that renders at any weight on a continuous axis, rather than one file per weight.

XSS

Cross-site scripting: dropping untrusted text into a page as HTML so that `<script>` in that text actually executes. Prevented by escaping.

Design tokens

Named, reusable values (colours, radii, widths) that encode a design system’s decisions. Here they are CSS custom properties in a `:root` block.

PolyForm Strict

A source-available licence granting permission to read the source and nothing else. Not open source.